

The Work a Verifier Needs

Certified Int8 Output Heads That Match a Production Kernel's Rounding

Samuel Mausberg

Preprint · Qwen3.5-4B in SGLang on one NVIDIA GH200

github.com/SamMausberg/verified-progress

AI (Claude Opus 5.5, GPT-6 Astra Pro) was used in developing the paper and the code.

A verifier needs one token, yet the engine scores 248,320

Speculative decoding: a drafter proposes tokens; the target checks them all in one forward pass.

A greedy verifier accepts a drafted token only if it is the target's top-scoring entry.

How little of the head can an exact shortcut compute?



one logit per vocabulary entry:
248,320 per position

Measured on recorded engine states, then served

Model and engine

Qwen3.5-4B with MTP-4B and DFlash-4B, SGLang at one commit, one GH200

Decisions

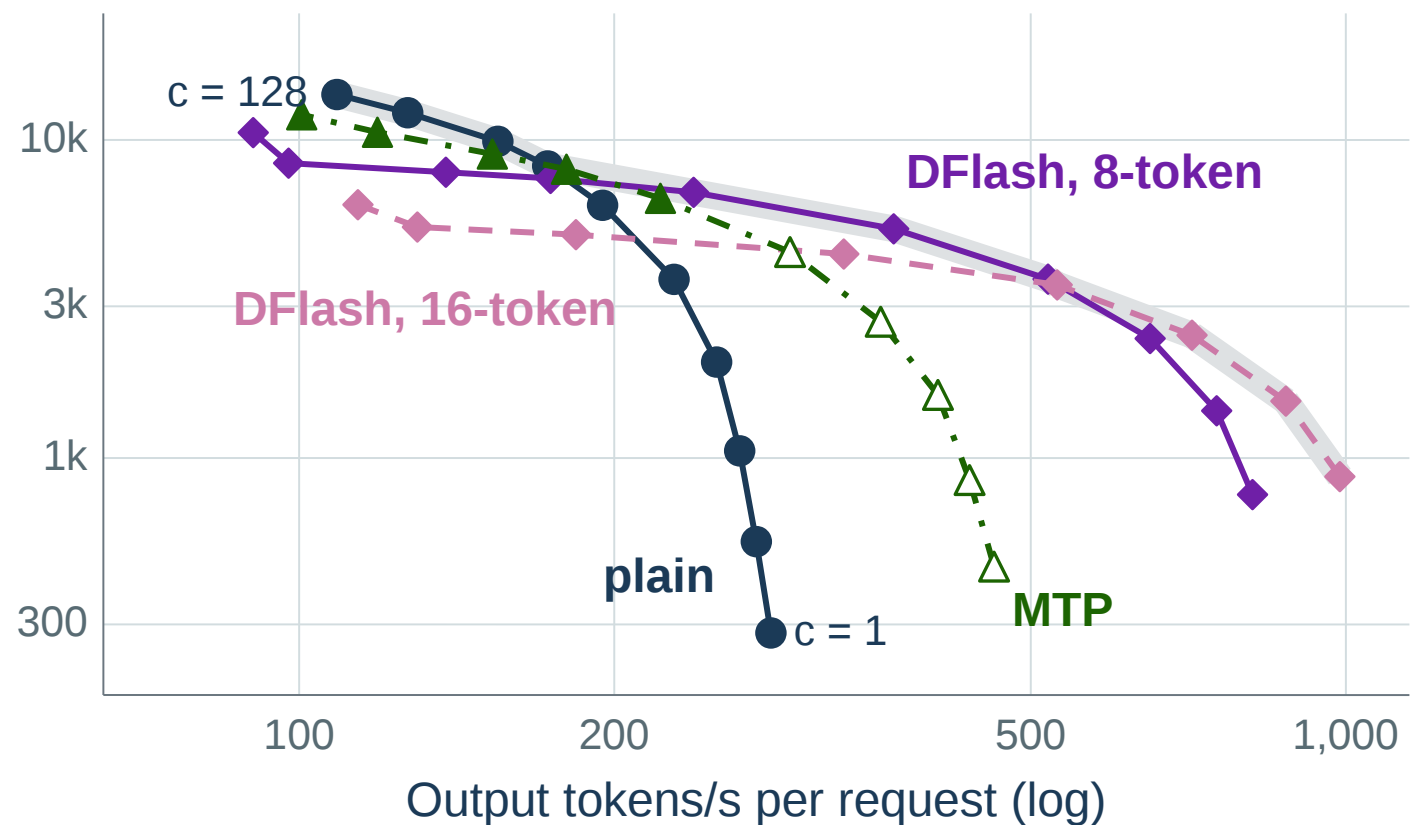
Captured head inputs replayed in FP64, in our kernel, and checked inside SGLang

Speed

$c = 1\text{--}128$ requests in flight, $1,152$ prompts $\times$ 512 tokens, three sessions per confirmed result

Each drafter's best arm beats plain decoding up to $c = 32$

Output tokens/s on the GPU (log)



3.06×

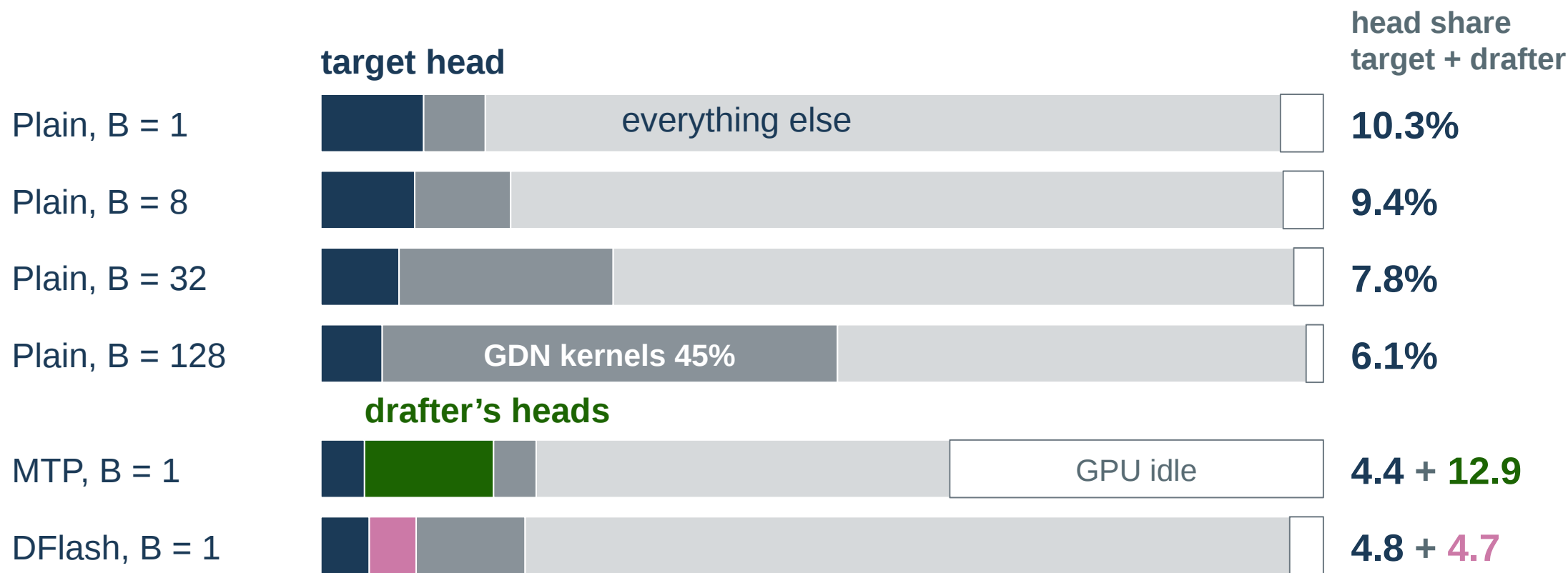
throughput at $c = 1$: 16-token
DFlash over plain decoding,
same attention

Mean accepted tokens per verify
cycle

DFlash-16	5.59–5.77
DFlash-8	4.66–4.81
MTP	3.24–3.28

Arm: a tuned serving configuration · grey band: the envelope, the best arm at each c · hollow: one session

The head is 10% of a plain step at batch 1, 6% at 128

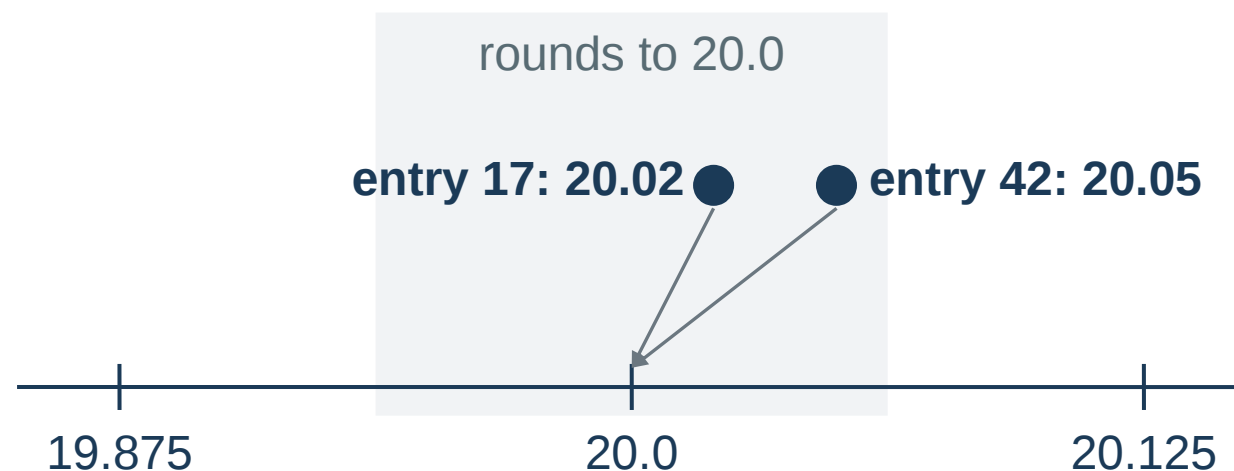


GDN: Gated DeltaNet, the linear attention in three of the model's four layers, with a recurrent state per request

A free head would make a batch-1 plain step only about 11% faster.

“Exact” must mean the engine’s token, not the exact winner

A made-up example



Exact winner: 42. The engine returns 17: both round to 20.0, and argmax takes the first index.

0.5%

of greedy positions: the exact winner is not the engine’s token

1.0%

the top two logits tie in BF16

Even stock SGLang changes its output with the batch

167 of 320

prompts' greedy outputs differ between 1 and 32 requests in flight on one stock server

First differences come from batch-dependent kernels upstream of the head

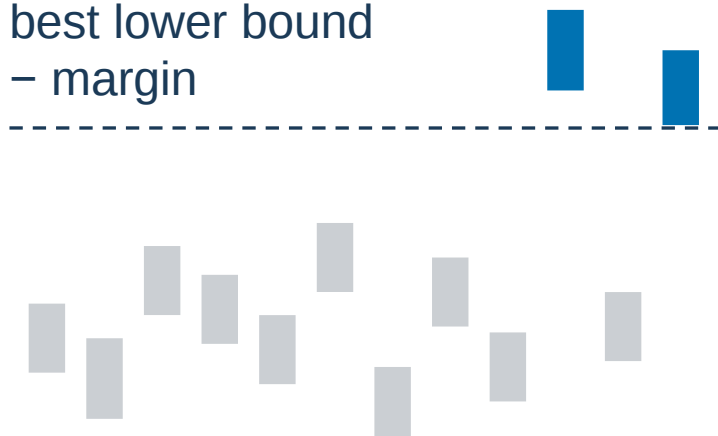
Never from the head GEMM or its tie rule

So every comparison fixes the batch shape and configuration

Bound every score, recompute the few that could win

1 Screen with int8

best lower bound
– margin



entries below the line
cannot win: dropped

2 Recompute a few

a



b



exact FP64 scores,
widened by the stock
kernel's error bound

3 Decide or fall back

one token under every
admissible accumulation:
return it

still open:
run the stock head

Schematic. The int8 copy reads half the head's bytes; the stock kernel's error bound is an assumed, tested model.

About 1.8 recomputations per decision, every token equal

1.8

entries recomputed per decision

median 1; kernel replay of 60,000 positions

0.28–1.46%

of positions left to the stock head

depending on the error model

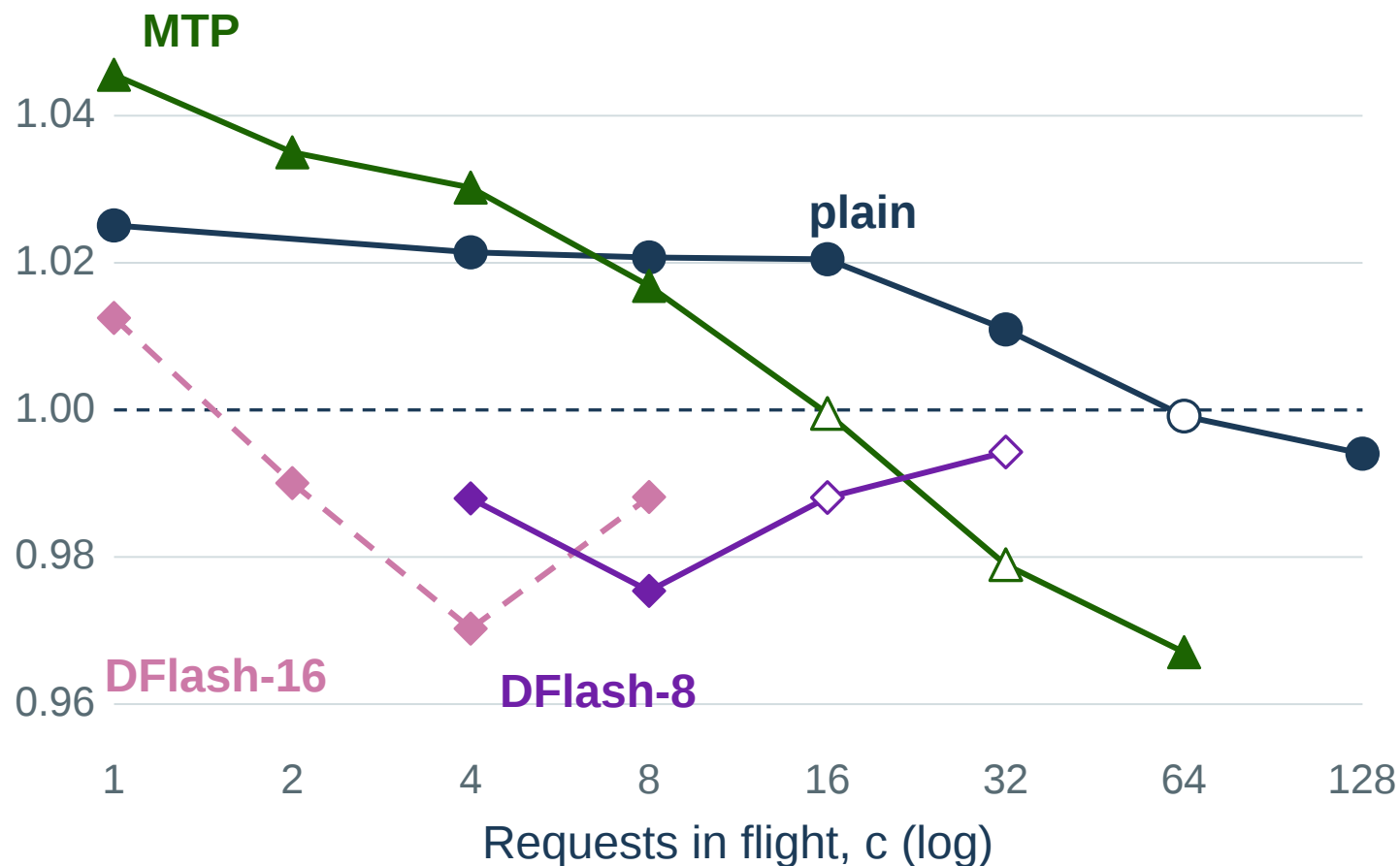
0 of 592,433

served positions differ from the stock head

check mode runs the stock head beside the certified one; every certified call counted

The head gains 1–5% at $c = 1$, slows the fastest arm at 2–8

Throughput with the head $\div$ without



+1.3–4.6%

at $c = 1$, 28–71% of the predicted gain

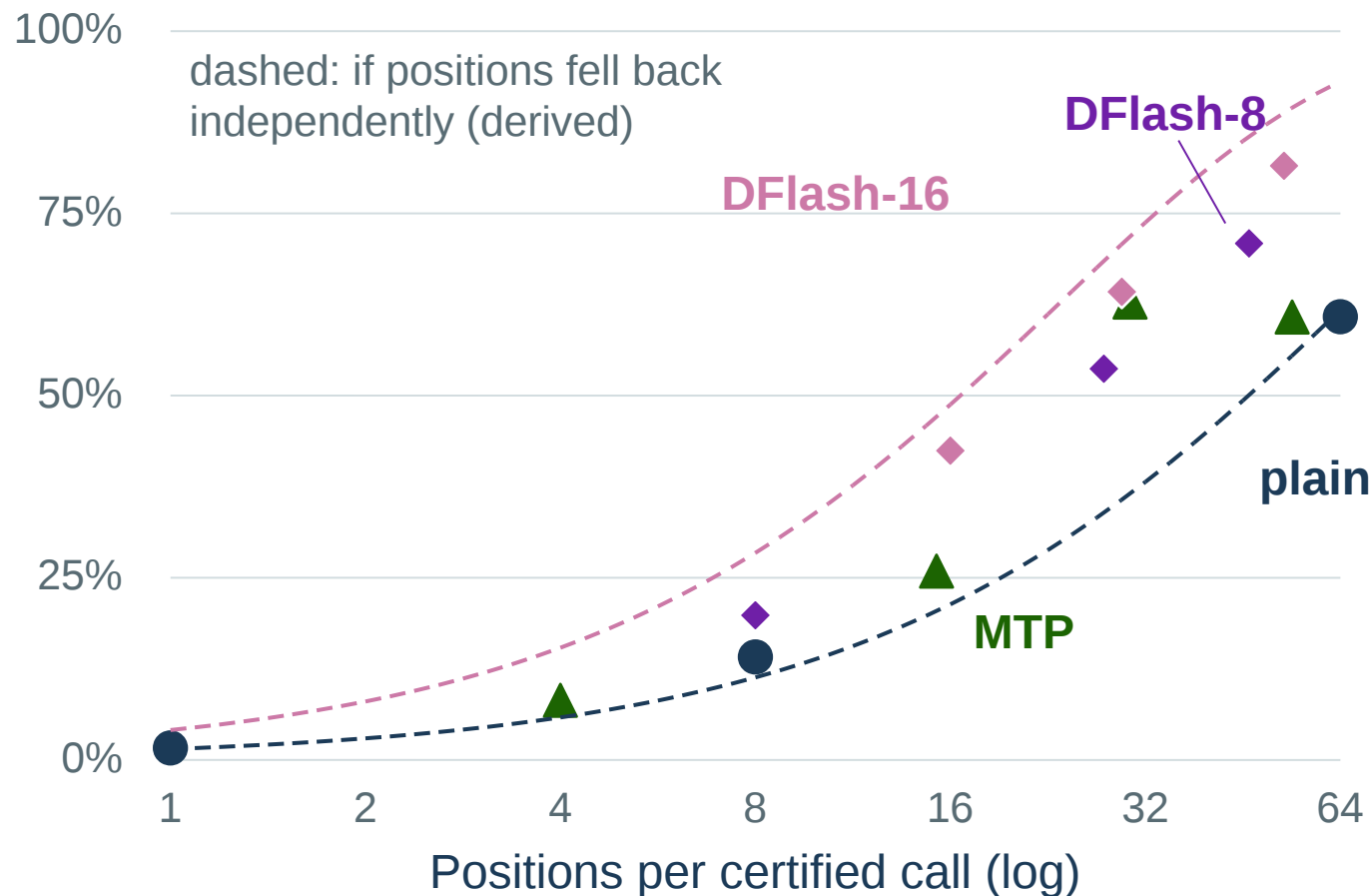
–1.0–3.0%

for the fastest arm at $c = 2–8$

Filled: 95% interval excludes 1.
Three sessions, each arm against itself.

Most DFlash verify calls above $c = 1$ need a fallback

Calls with a fallback (check mode)



54–82%

of DFlash verify calls at $c = 2-8$ need a fallback

0.65×

the stock head's time at one position: the int8 pass is not memory-bound

Fallback cost inside the served graph: not timed

Transport needs drift below 0.01; median drift is above 0.9

draft tile maximum shift + allowance target bound known target score

6 → **+2 + 0.5** → **8.5** → **9: skip the tile**

Made-up numbers: the drafter's scores, carried to the target's head input by a Cauchy–Schwarz bound

Threshold

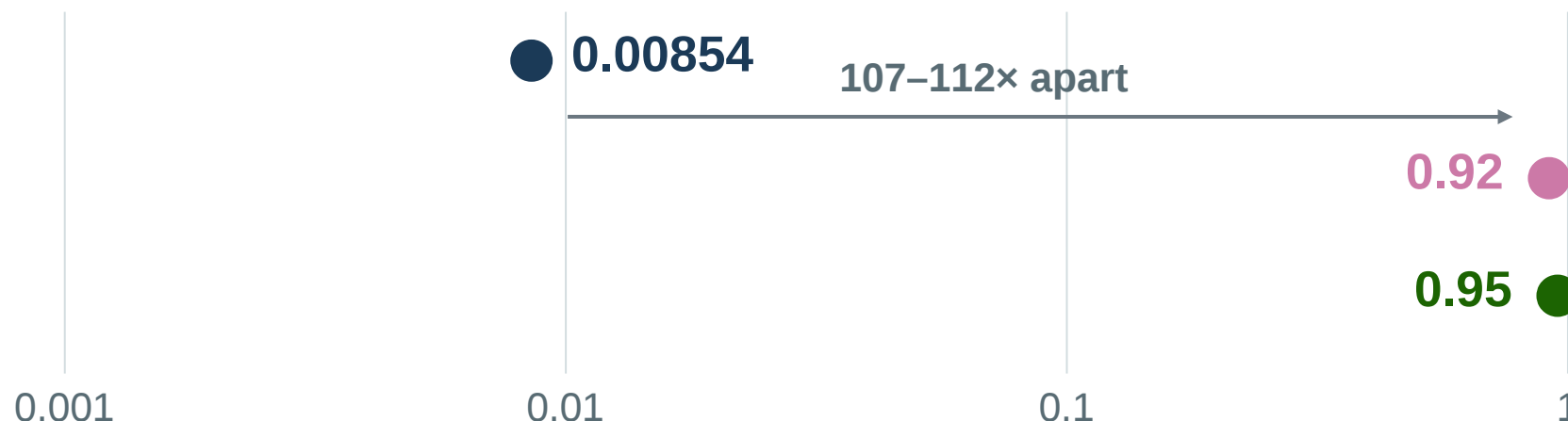
median over entries

DFlash-4B

median drift, 40,000 pairs

MTP-4B

median drift, 40,000 pairs



Drift: $\|h_{\text{target}} - h_{\text{draft}}\| \div \|h_{\text{target}}\|$ (log)

Certified transport skips at most 0.59% (DFlash-4B) and 0.64% (MTP-4B) of the vocabulary.

Engine changes outside the head gained more

fold: SGLang’s fold-every-commit GDN verify, without per-position state snapshots · FA4: FlashAttention 4

Certified head

three arms · c = 1 · 3 sessions



Replayed GDN decoding

plain · c = 128 · one run pair



Fold, narrow tiles, FA4

DFlash-16 · c = 1–4 · per request · 3 sessions



Fold, FA4 target

DFlash-8 · c = 8–32 · 3 sessions



Prefill delay and cap

MTP · c = 48–128 · 3 sessions



Light: up to the smallest gain; dark: the range across c or arms. Each against the tuned arm it changes; below the line, against the best arm without speculation.

Five other approaches missed a rule declared in advance

Lossy

FP8 decode rate $1.014\times$ at $c = 64$ (rule 1.05); int4 fails the probe, $0.715\text{--}0.870\times$ above $c = 1$;
FP16 state $1.0\text{--}1.4$ GSM8K points below (budget 1)

Exact, but slower

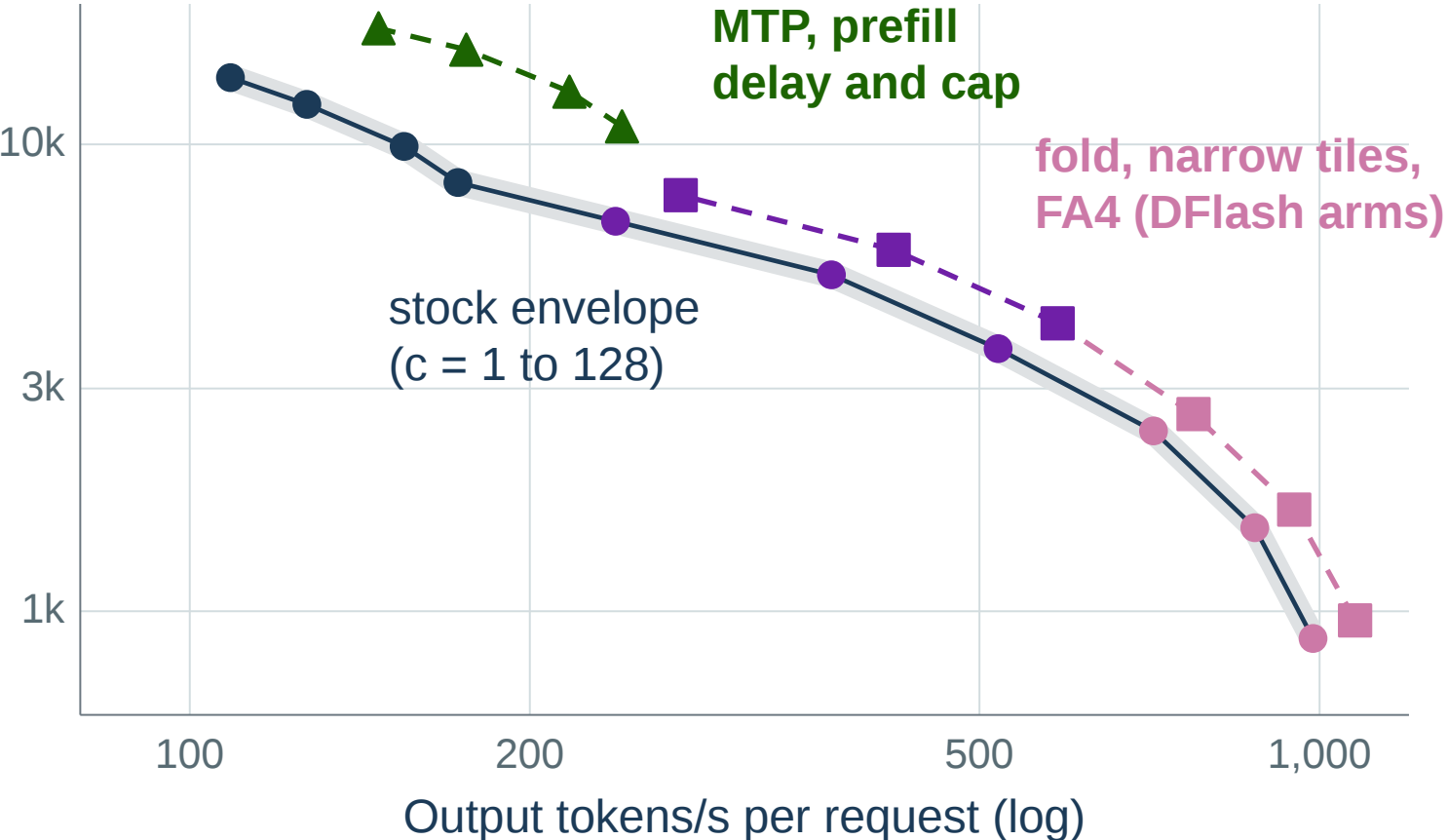
Fold + routing + certified head: $0.972\times$ at $c = 2$ (declared ≥ 1.007)

Kernel

Split-KV verify kernel saved 7.6 of the $200\text{ }\mu\text{s}$ its rule required

Both confirmed changes plot beyond the stock envelope

Output tokens/s on the GPU (log)



+8–9%

per request, $c = 1-4$

+12–14%

throughput, $c = 8-32$

1.18–1.32×

the best plain arm at $c = 48-128$; first tokens later

Ratios within each confirmation's sessions; the plot combines sessions.

Where compared, outputs match stock up to rounding

Greedy outputs against stock,
same configuration

Certified head identical; **fold** bitwise;
fold + FA4 and **prefill delay** up to
rounding

GSM8K, 1,319 problems, sampled

-1.36 to +1.06 points

for two MTP arms, 8-token DFlash and
replayed GDN plain decoding, not the
changes on the left

**Rules out only losses above about 3.3
points**

Hold the fast path to the engine's own token

A fast path held to the engine's own output, with its kernel as fallback, is not specific to the head.

Limits

Error model tested on one GPU, not proved; fallbacks inside the served graph not timed; one model, engine commit, GPU and workload

Next

Time the in-graph fallbacks; a delay-only admission arm; batched equality for fold + FA4

Backup slides, by question

Is it really exact? The contract and the error model	B02–B09
Why is the served gain so small?	B10–B14
Why does transport fail?	B15–B18
The engine changes and their exactness classes	B19–B24
The profile in detail, related work, upstream fixes, code	B25–B29

Certify the stock kernel's token, or run the stock kernel

Bound each logit's FP32 sum, round the bounds to BF16, then:

Every admissible sum gives the same first maximum → **Return that token**

The bounds leave the token open → **Run the stock head**

Whole-batch fallback: the stock matmul at the same M; no error model

Per-position fallback: the open position's candidates; needs batch-invariant logits (held at 20 batch sizes)

The error model

Conservative: 2D roundings of 2^{-23} , any order

Hopper: tensor-core blocks of 16, 25 bits, truncation, plus split-K

Tested: 14 cuBLAS kernels on crafted inputs; FP32-output error 1/58 of the bound

Not covered: BF16 split-K partial sums

The reference is the engine's actual result

Reference	Computation
R-real	Exact dot products
R-bf16	Exact dot products → BF16 rounding
R-stock	Existing kernel → BF16 rounding

Same inputs, batch shape and configuration.

Assumed error models are not vendor guarantees.

Fixed ties certify; uncertain ties fall back

Case	Token 17	Token 42	Action
Separated	1.0	0.99609375	Return 17
Fixed tie	1.0	1.0	Return 17
Uncertain tie	1.0	1.0 or 1.0078125	Fallback

Rounded-score bounds for a candidate k:

Earlier j: $\text{upper}(j) < \text{lower}(k)$ Later j: $\text{upper}(j) \leq \text{lower}(k)$

Check mode found no differing token in 592,433 positions

0 of 592,433

checked positions differ from the stock head run beside it

1.03–1.21×

closed-loop divergence, certified vs stock, over stock vs stock

3.8–25.7 GB

more memory for the certified CUDA graphs

0.6–1.2%

cost when gated off, at calls above 64 positions

Every exact change matches stock up to rounding-level ties

Same configuration	Host-gap patches: tokens identical on 192 of 192 requests Fold verify (matched pools): bitwise; with FA4: ties, one BF16 step or near ties ($c = 1$) Certified head: 0 of 592,433 positions differ in check mode
Closed loop	Batches evolve differently run to run: certified vs stock diverges at 1.03–1.21× the stock-vs-stock rate
Task accuracy	GSM8K, sampled: exact arms within −1.36 to +1.06 points of plain decoding; it cannot exclude losses under about 3.3 points

SGLang’s typical BF16 error matches Hugging Face’s

Mean |log-probability error| on FP32’s top token (nats); misses above 2 nats

Implementation, path	Sample 1: 6,144	Sample 2: 9,216	Misses > 2 nats
SGLang (pin, unpatched) · decode	0.0054	0.0039	0
SGLang (pin, unpatched) · prefill	0.0054	0.0045	1
transformers, torch GDN · decode	0.0057	0.0044	0
transformers, torch GDN · prefill	0.0056	0.0049	1
transformers, fla GDN · decode	0.0056	0.0042	0
transformers, fla GDN · prefill	0.0056	0.0047	1

The 8.9-nat miss is an SGLang outlier at one position; a second position is ill-conditioned: FP32’s own top token flips under rounding-size noise. No issue filed.

Quality results do not establish equivalence

GSM8K · 1,319 problems	Accuracy
Plain run 1 / run 2	89.99% / 90.30%
Buffered MTP	89.54%
MTP, snapshot verify	88.93%
DFlash, block 8	89.69%
Replayed GDN plain decoding	91.05%

Paired 95% intervals allow $\approx$ 3.3 percentage points of loss.

Seeded sampling needs fixed noise per token

z_i : score · $T > 0$: temperature · G_i : Gumbel noise

Choose the largest $z_i / T + G_i$.

Noise key: seed, sequence index, token ID

Uncertain choice → stock sampler

With its fallback it matches SGLang's seeded sampler: 60,000 replayed positions at $T = 0.7$ and 1, and 21,016 in SGLang at $T = 0.7$

Acceptance against the stock rejection sampler: not measured

Alone, the int8 head takes about 0.7 of the stock head's time

Head alone, one call of M positions

One position, measured

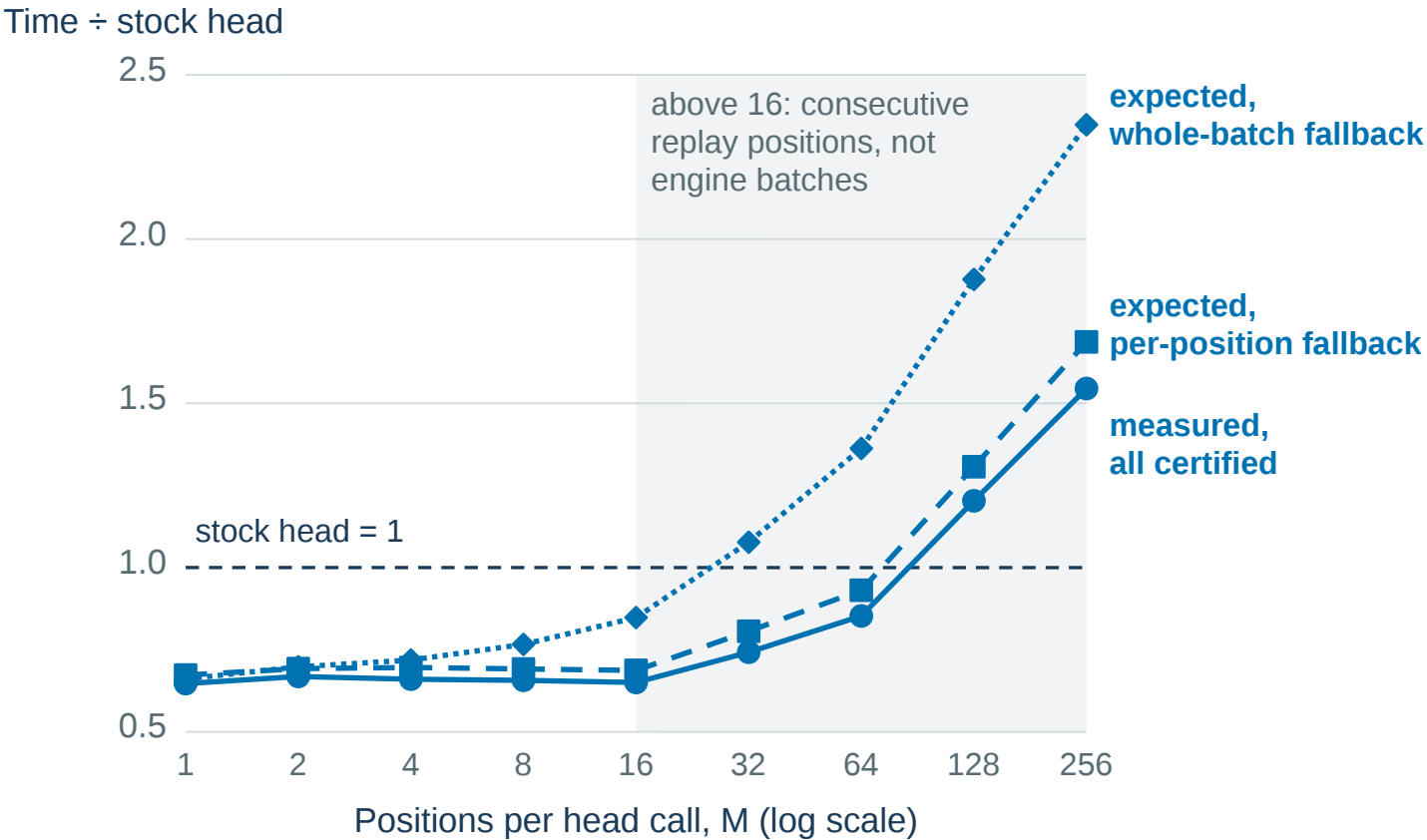
370.5 μ s

stock head: GEMM, FP32 copy, argmax

239.6 μ s

certified head, every position certified

With the head at 10.3% of a batch-1 plain step, a step would shorten by at most 3.4–3.5% (derived); served, plain decoding gained 2.5%.



Int8 leaves few candidates; fallback grows with the batch

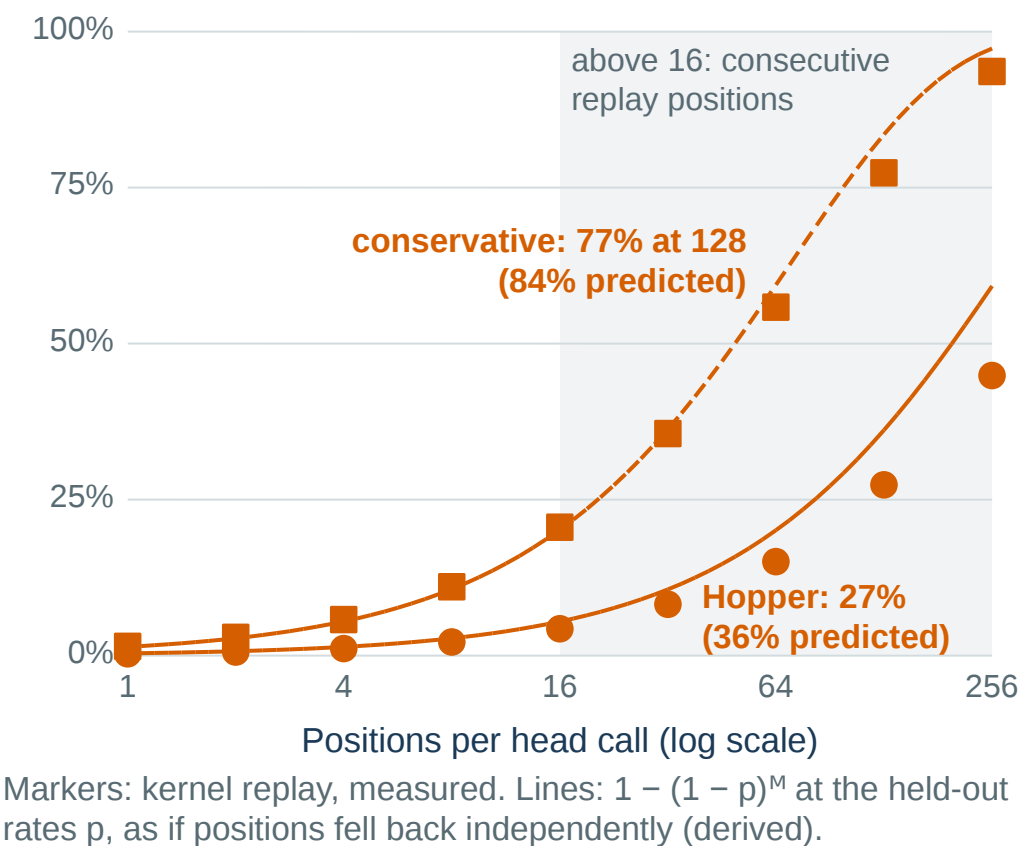
Candidates per greedy decision, by copy of the head (held-out replay, 6,005 positions, exact reference)

Copy	Bytes	Mean	p99	Max
Int8, per entry	0.50	1.55	8	30
Int8, 128 columns	0.52	1.34	5	23
Int8, 32 columns	0.54	1.26	5	20
FP8 E4M3, per entry	0.51	4.89	60	605
Int4, 128 columns	0.27	72,847	248,295	248,320

Why int8 is enough: its intervals are about two logits wide; the median gap between the two largest logits is 7.3 logits.

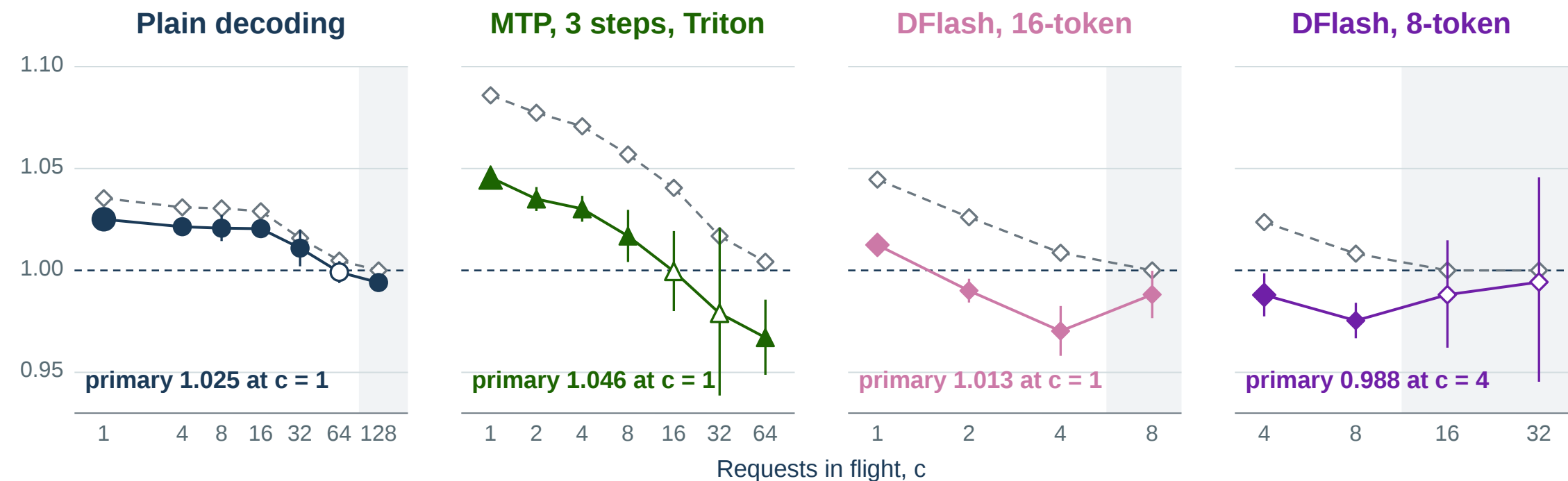
Held-out fallback: 0.35% (Hopper model) and 1.40% (conservative) of positions under the interval condition; the gap condition never goes below about 2%.

Calls with at least one open position, which the whole-batch fallback sends to the stock head



Every served arm fell short of its prediction

Throughput with the certified head ÷ without, each tuned arm against itself



Filled: 95% interval excludes 1 · bars: 95% intervals · grey ◇: prediction · shaded: head gated off

At c = 1 the gain is 71% (plain), 53% (MTP) and 28% (16-token DFlash) of the prediction, tokens identical; gains continue to c = 32 (plain) and 8 (MTP); both DFlash arms lose 1.0–3.0% where the head is active.

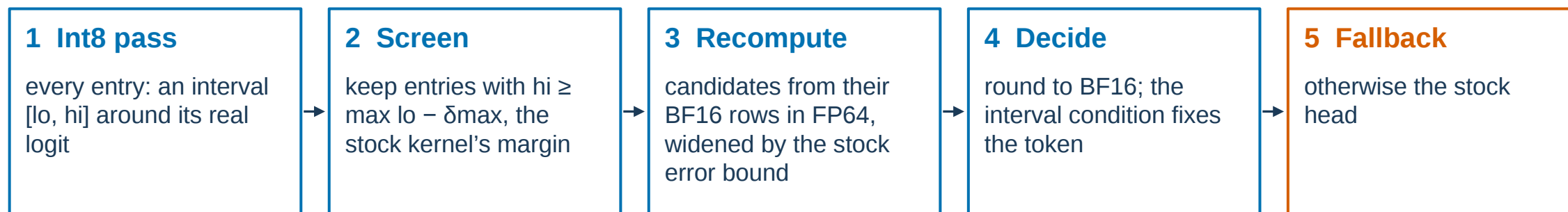
Certified head served: every point against its prediction

Throughput ratio, certified ÷ stock; second line: predicted before the runs

Arm	c = 1	c = 2	c = 4	c = 8	c = 16	c = 32	c = 64	c = 128
Plain decoding	1.025 <i>1.035</i>		1.021 <i>1.031</i>	1.021 <i>1.030</i>	1.020 <i>1.029</i>	1.011 <i>1.016</i>	0.999 <i>1.005</i>	0.994 <i>1.000</i>
MTP, 3 steps	1.046 <i>1.086</i>	1.035 <i>1.077</i>	1.030 <i>1.071</i>	1.017 <i>1.057</i>	1.000 <i>1.041</i>	0.979 <i>1.017</i>	0.967 <i>1.004</i>	
DFlash, 16-token	1.013 <i>1.045</i>	0.990 <i>1.026</i>	0.970 <i>1.009</i>	0.988 <i>1.000</i>				
DFlash, 8-token			0.988 <i>1.024</i>	0.975 <i>1.008</i>	0.988 <i>1.000</i>	0.994 <i>1.000</i>		

The predictions assumed plain decoding’s fallback rate on every path; drafted positions fall back more.

Int8 screens every entry; a few are recomputed exactly



The error bound of the copy

$$\beta = \|e\| \|h\| + \gamma \|\bar{w}\| \|h\| + \eta'$$

e: the copy's known error per entry; γ : tensor-core accumulation (conservative model).

The term $\|e\| \|h\|$ assumes the worst alignment of e and h. Its median on captured states is 1.00 logit, about 75 times the realized error (0.013).

The copy is 0.64 GB beside the BF16 head; one pass reads half the head's bytes.

The kernel, and seeded sampling

A W8A16 GEMM computes each interval in its epilogue with directed rounding and keeps, per chunk, the four largest upper ends; one CUDA graph per batch size holds it and the fallback.

Each compiled variant self-tests before it certifies; every call checks eight exact entries per position. The checks caught two faults, both refused.

The same steps on Gumbel-noised scores certify SGLang's seeded sampler's token: 60,000 replayed positions at $T = 0.7$ and 1, 21,016 in SGLang at $T = 0.7$.

Drafts agree with the target on tokens, not on head inputs

50.6 vs 159

DFlash-4B's head input norm is always $\sqrt{2560}$ (unit-gain final RMSNorm), a third of the target's median

Norms alone give drift ≥ 0.68

0.91

drift left after the best rescaling of the draft input (median cosine 0.41)

The angle is large too

0.91 / 0.99

median drift where the draft token was accepted (DFlash-4B / MTP-4B)

Same token, different head input

Per position, draft and target logits differ by 2.0–2.1 RMS (medians): draft scores are not small perturbations of target scores.

The int8 copy leaves about 1.6 candidates on the DFlash-4B verification positions a greedy verifier needs.

What would rescue it: a drafter whose head input tracks the target's. Scaling the measured drift down 10× would let the same bounds skip 49% (DFlash-4B) of one position's vocabulary; 100×, 84% (a model extrapolation). Batches would still need most of it: even oracle radii need 97% for random groups of 16 DFlash-4B pairs.

Transport's uncertainty is radius × change in head input

Tile geometry

w_i : entry's weight vector

μ : centre; r : radius

$$\|w_i - \mu\| \leq r$$

State change

Δ = target – draft

M : draft tile maximum

$$\text{Target maximum} \leq M + \mu \cdot \Delta + r\|\Delta\|$$

Skip only when this bound is below a known target score.

Changing the tiles did not rescue transport

Vocabulary excluded · T: transport · S: static bound without draft scores

Entries / tile	DFlash T	DFlash S	MTP T	MTP S
Contiguous 64	0.08%	0.08%	0.08%	0.08%
Contiguous 256	0.00%	0.00%	0.00%	0.00%
Contiguous 1,024	0.00%	0.00%	0.00%	0.00%
Random 256	0.00%	0.00%	0.00%	0.00%
k-means 64	0.59%	0.58%	0.64%	0.64%
k-means 256	0.10%	0.10%	0.11%	0.11%
k-means 1,024	0.02%	0.02%	0.02%	0.02%

DFlash: 4,020 pairs · MTP: 16,016 pairs

Transport leaves over 99% of vocabulary entries to check

DFlash

99.4% remain



MTP

99.4% remain



Little benefit beyond bounds without draft scores.

Best tiles on the two tested drafters

Every change against the arm it modifies

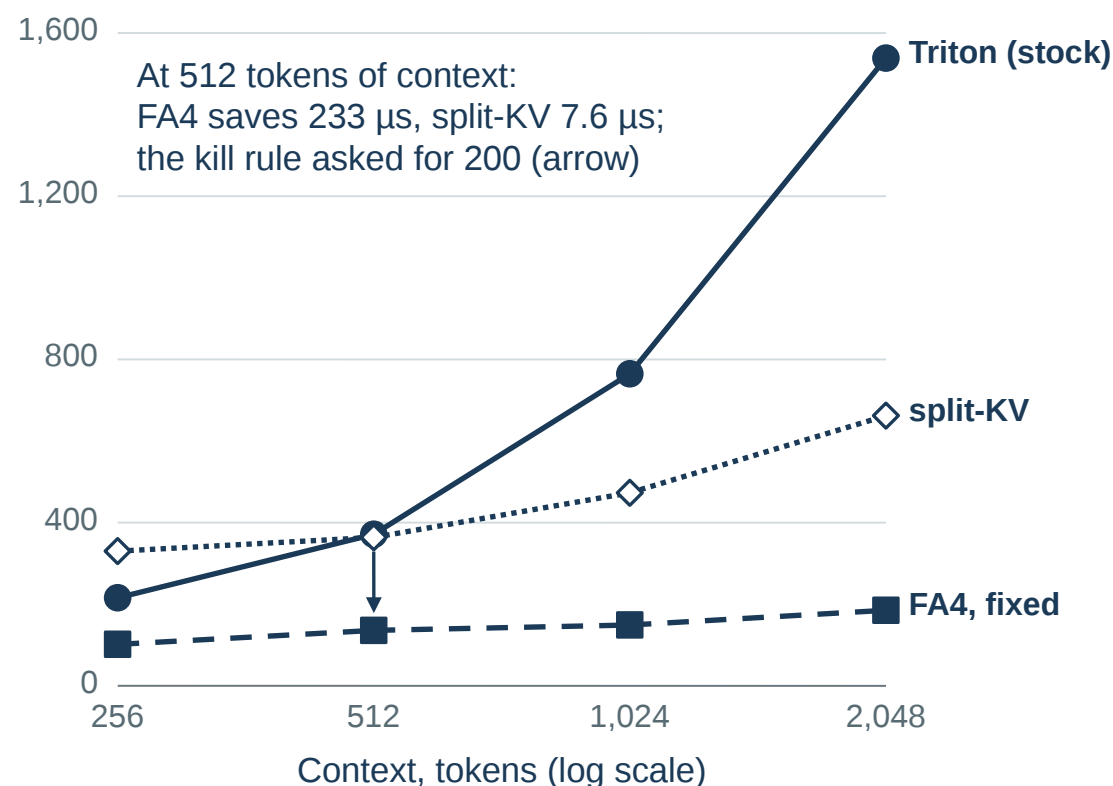
Ratio with ÷ without; brackets: run-pair range, session range or 95% interval

Change	Arm	c	Ratio	Sess.	Outputs
Host-gap patches	MTP	1–8	1.094–1.096 (1.090–1.101)	1	token-identical
Prefill delay, cap 16	MTP	32–128	1.19–1.48 (1.17–1.50)	3	rounding (64, 128)
Backbone GEMM routing	plain	1	1.035 (1.034–1.035)	1	rounding
Replayed GDN decoding	plain	128	1.081 (one pair)	1	rounding
Fold verify	DFlash-16	1 · 8–32	0.968 · 1.061–1.120	1	bitwise
Fold, narrow tiles, FA4	DFlash-16	1–4	x 1.084–1.093 (1.056–1.114)	3	rounding (c = 1)
Fold, FA4 target	DFlash-8	8–32	1.124–1.137 (1.108–1.166)	3	rounding (c = 1)
Certified head	plain · MTP · D-16	1	1.025 · 1.046 · 1.013	3	identical
Certified head	D-16 · D-8	2–4 · 4–8	0.970–0.990 · 0.975–0.988	3	check mode

“rounding”: exact up to rounding, as classed at the c given. The prefill delay against the best non-speculative arm: 1.18–1.32× at c = 48–128.

The fold, narrow tiles and FA4: 8–14% at c = 1–32

Target attention, 8 layers, one request: μs per forward (microbenchmark)



80 // 128 = 0

Qwen3.5's head-dim-256 tile on sm_90 is 128×80 : floor division gave 128 loader threads no page-table entries, so FA4 never compiled. The fix is in flash-attention and in SGLang PR #35757 (open); we backported it.

1.084–1.093

per request, 16-token DFlash, c = 1–4: fold, narrow tiles, FA4 for drafter and target

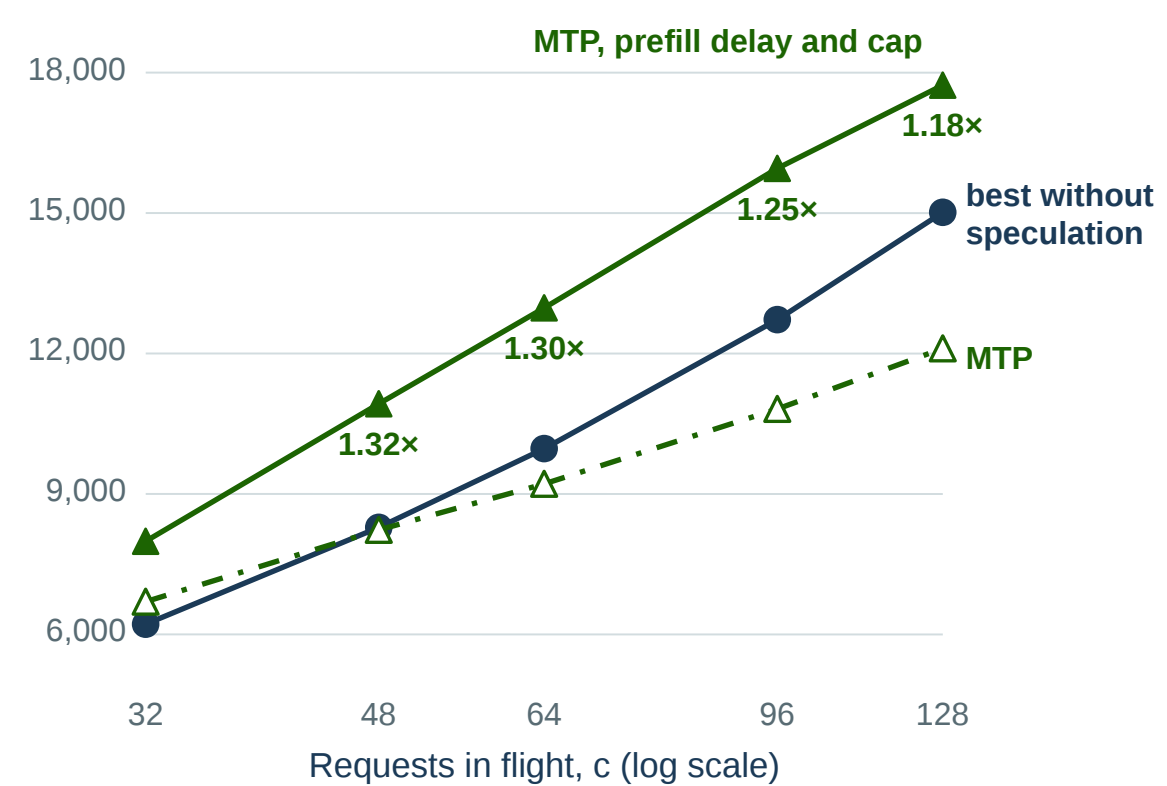
1.124–1.137

throughput, 8-token DFlash (already drafting with FA4), c = 8–32: fold, FA4 target

Exact up to rounding as classed at c = 1 (the fold alone bitwise); batched outputs not compared. Alone at c = 1–4 each FA4 change is a speedup (drafting 1.025–1.032, target 1.030–1.038 per request); at 4 neither is shown to shorten the cycle. The fold alone at 4: 1.005 (0.990–1.021).

A capped prefill delay puts MTP ahead at c = 48–128

Throughput, mean of three sessions; labels: × the best arm without speculation



33 ms

per lone prefill, 9 ms of it GPU work. Speculative requests finish one or two at a time and are prefilled alone; plain decoding's equal-length requests finish together and share a prefill.

1.18–1.32×

the best arm without speculation, every session; prefill batches at c = 128: 518 → 81

2.6–3.1×

plain decoding's 99th-percentile time to first token, at c = 48–128 (385–635 ms)

Exact up to rounding against MTP without it, classed at c = 64 and 128. At c = 1 and 8 it passes the no-harm rule (0.994, 0.992), but median time to first token rises 0.6–2.7 ms in every session pair. One setting, not tuned; no arm separates the delay from the cap.

Both declared approximations miss the quality budget

Budget: GSM8K within 1.0 point; logit probe top-1 ≥ 0.98 , KL ≤ 0.01

Approximation	Throughput vs best non-lossy	Quality	In budget
FP16 recurrent state	1.16–1.17× at c = 64–256	GSM8K –1.0 to –1.4 points	No
Int4 target and drafter	0.715–0.870× above c = 1	probe fails: top-1 0.969–0.975	No

FP16 state halves the recurrent state; its GSM8K loss is within the exact arms' own spread (–1.36 to +1.06), which this check cannot resolve.

Int4: the W4A16 GEMM is slower than BF16 from 32 rows per call, and the int4 drafter accepts fewer tokens.

The first composition fell below its declared range

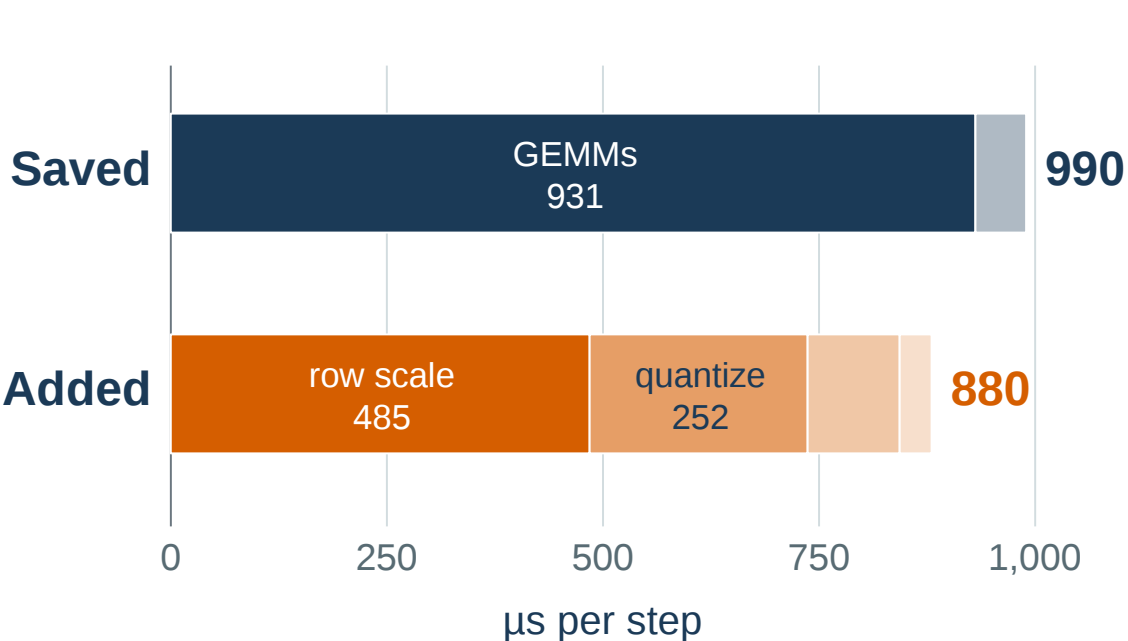
Against tuned 16-token DFlash: ratio and 95% interval

Arm	c = 1	c = 2	c = 4	c = 8
Fold + routing + certified head	0.986 0.982–0.989	0.972 0.966–0.979	1.013 1.007–1.020	1.079 1.063–1.095
Fold verify	0.970 0.967–0.974	0.987 0.984–0.989	1.009 1.002–1.016	1.073 1.065–1.081
Routing table	1.009 1.004–1.014	0.988 0.985–0.991	1.017 1.011–1.022	1.008 0.995–1.021
Fold + routing	0.974 0.967–0.982	0.971 0.959–0.984	1.021 1.008–1.033	1.082 1.058–1.107
Composed tree, switches off	0.999 0.996–1.003	0.998 0.995–1.002	0.999 0.990–1.009	0.995 0.983–1.007

The later composition, the fold with narrow tiles and FA4, gained +9.3% per request at c = 1.

FP8 halved the weight bytes; added kernels took it back

Plain decoding, one request: μs per decode step, FP8 with per-row scales against BF16 (Nsight)



1.014x

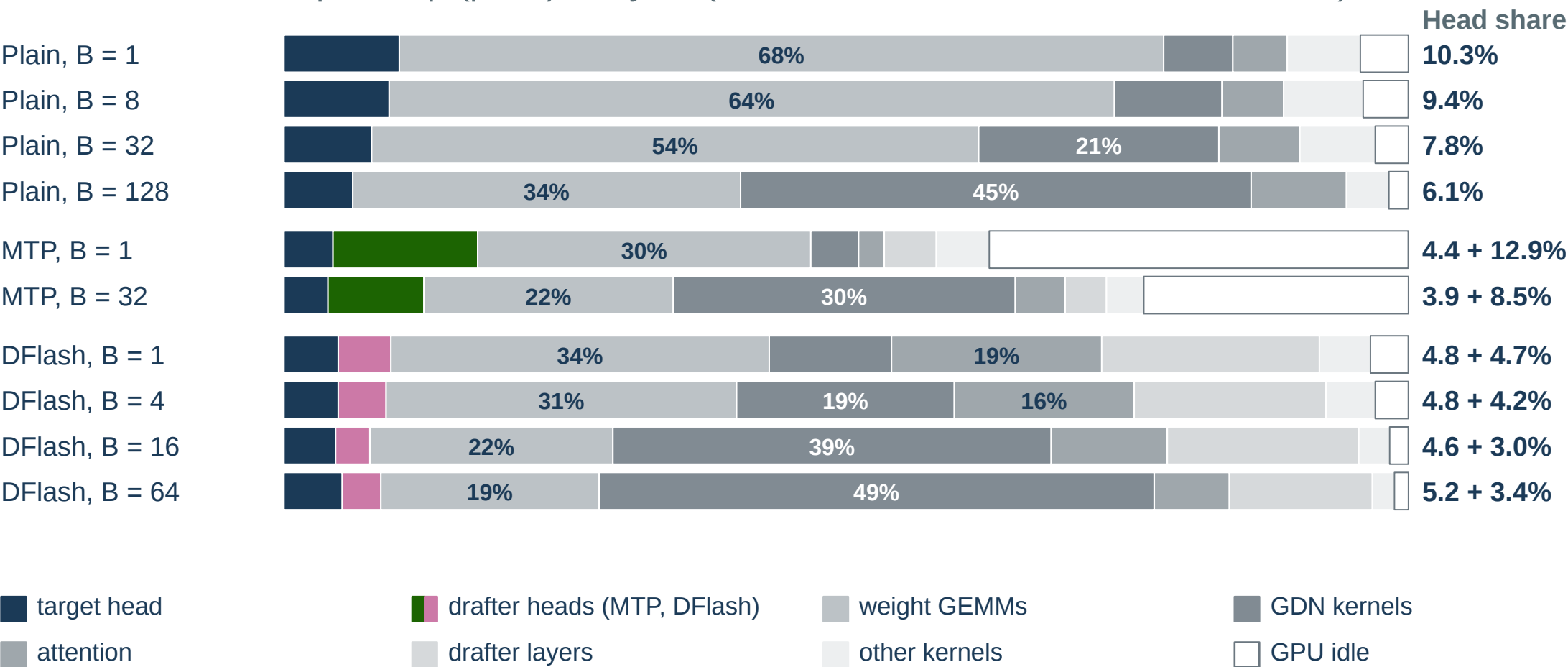
decode rate at $c = 64$ against a kill rule of 1.05

1.120–1.260x

without the row-scale kernel, at $c = 1\text{--}64$;
neither variant shown inside the quality budget

Weights dominate at batch 1, recurrent state at batch 128

Share of GPU time per step (plain) or cycle (MTP, untuned; 16-token DFlash, tuned)



New here: certifying a closed-source kernel's token

Prior art

Certified low-precision heads: Sparkpipe, dgpp, a Metal engine that replicates its kernel, knlp

Bound-and-refine search; cluster bounds (the static screen)

Sampling from bounded scores; drafter-invariant verification

This paper

A contract with a closed-source kernel's rounding, under a tested error model, with the stock head as fallback

Transport of draft-side tile summaries: no precedent found, and a negative result

Seven pull requests and six reports upstream, all open

Item	What it fixes or reports	Status
flash-attention #2957, #2958	FA4's SM90 paged-KV loader copies rows past a partial tile (head dims 80, 96, 160, 224); the PR skips those copies	Open, no reply Open, no review
Comment on SGLang #35757	Confirms the head-dim-256 compile fix on Qwen3.5; reports the loader bug with the vendored-copy diff	Open, no reply
Comment on SGLang #42019	max(1, floor) left tile_n 144 short. The author adopted the ceiling division; on an H200 it alone does not fix tile_n 144. We pointed to #2958 and offered the diff	Open; author replied, we answered
SGLang #42263	Builds sm_90a for the aarch64 sgl-kernel wheel, whose SM90 CUTLASS GEMMs computed nothing on GH200	Open, no review
SGLang #42209, #42195	The fold for DFlash and MTP; FlashInfer planning without blocking reads (both opt-in)	Open, no review
SGLang #42160, #42126, #42094	64-bit offsets in a fused kernel; the packed GDN projection on CUDA; a tied MTP drafter start-up crash	Open, no review
Issue #42085; on #41351, #38118	GEMV flag ignored by unquantized layers; radix-cache history changes log-probabilities; a chunked tail counted twice	Open, no maintainer reply

One repository holds the code, harness and evidence

src/certified_head/

the int8 head: Triton kernels, bounds, self-tests

engine/sglang/patches/

opt-in SGLang patch series, one per topic

bench/

serving harness: arms, launch checks, sweeps, Pareto

experiments/, evidence/

scripts; committed results, each README with its command

tests/, formal/

exact CPU references, pytest, Lean 4 lemmas

CPU checks are separate from GPU campaigns

github.com/SamMausberg/verified-progress

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements-cpu.txt \
    -r requirements-dev.txt
python scripts/verify_artifact.py
python tests/test_precision.py
python tests/test_contracts.py
python tests/test_state_structure.py
python -m pytest tests/
```

GPU procedures and pinned environment: see speaker notes.